Exercise: Publish a Cars CRUD Docker Image with GitHub Actions

Goal: Extend your existing `.github/workflows/tests.yml` so it still tests pull requests to `main` and, **after the tests pass on a push to `main`**, builds the Cars CRUD Docker image and publishes it to your Docker Hub account.

This is a **write-it-yourself exercise**, with requirements and hints rather than a complete finished workflow.

Prerequisites: You have completed TESTING_WORKSHOP.md, GITHUB_ACTIONS_GUIDE.md, GITHUB_ACTIONS_EXERCISE.md, and DOCKER_GUIDE.md. Your existing `tests.yml` already runs unit, integration, API, and E2E tests successfully for pull requests. You have a Docker Hub account and a `cars-crud` repository there.

1. What the finished workflow should do

Event	Expected result
Open or update a PR targeting `main`	Run all four test groups. **Do not** publish an image.
Merge that PR into `main` (or push to `main`)	Run all four test groups. If they pass, build and publish the image.
Tests fail on `main`	Mark the test job failed and **skip publishing**.

Why not publish from pull requests? Publishing uses a Docker Hub access token. Keep that credential out of jobs that execute unmerged pull-request changes. Also, GitHub does not normally provide repository secrets to workflows triggered by pull requests from forks.

2. Docker Hub and the course's Azure pull proxy

When working **locally in China**, pulls from Docker Hub may be slow. The course provides an Azure intermediary at **`efreiwut.azurecr.io`** for supported cached images:

```
# Normal Docker Hub pull
docker pull python:3.12

# Equivalent image/tag through the course proxy
docker pull efreiwut.azurecr.io/cache/library/python:3.12
```

A Dockerfile can likewise use either base-image path:

```
FROM python:3.12
# Alternatively, for local access through Azure:
```

```
# FROM efreiwut.azurecr.io/cache/library/python:3.12
```

The Cars CRUD app uses **`python:3.12-slim`**, so the matching Azure path ends in **`/python:3.12-slim`**. In the Docker guide, the Dockerfile accepts `BASE_IMAGE` as a build argument: local builds can select the Azure proxy, while the default remains the public Docker Hub image.

For this exercise:

- **Local development:** use the Azure path if it improves pulling the Python base image.
- **GitHub Actions:** the hosted runner can ordinarily pull `python:3.12-slim` from Docker Hub, so **no proxy setting is required**. The workflow can also use the Azure proxy if it is accessible and intentionally configured.
- **Publishing:** push the finished Cars CRUD image to **your Docker Hub repository** (`YOUR_DOCKER_ID/cars-crud:latest`), not to `efreiwut.azurecr.io`. The course proxy is not a substitute for your Docker Hub publishing credentials, and it is not guaranteed to proxy personal Docker Hub repositories.
- **Production:** organizations often use their own registries and image URLs, such as `registry.example.com/team/cars-crud:v1`.

If your Dockerfile uses the build argument from the guide, no `build-args:` setting is needed in GitHub Actions for the standard Docker Hub base. The proxy is only an alternative source for pulling the base image.

3. Prepare your exercise branch

From your existing `cars-crud/` repository:

```
git switch main
git pull --ff-only origin main
git switch -c docker-image-workflow
```

Make sure the following files are present in this branch:

```
cars-crud/
├── Dockerfile
├── .dockerignore
├── requirements.txt
├── app/
└── .github/
    └── workflows/
        └── tests.yml
```

If you have not committed `Dockerfile` and `.dockerignore` yet, add them using the Docker guide. **The Dockerfile and application files must be committed** so GitHub can build the same image you built locally.

4. Configure GitHub repository secrets

1. In Docker Hub, create a **personal access token** with **Repo Read & Write** permissions if you do not already have one.
2. In your GitHub repository, open **Settings > Secrets and variables > Actions**.
3. Under **Repository secrets**, choose **New repository secret** and create:

Secret name	Secret value
`DOCKERHUB_USERNAME`	Your Docker Hub ID / username.
`DOCKERHUB_TOKEN`	Your Docker Hub personal access token.

The username could also be stored as a GitHub variable, but use the two names above for consistency in this exercise. Do **not** put the token in the YAML, in a committed `.env` file, or in a terminal screenshot.

5. Workflow requirements

Modify the existing file: `.github/workflows/tests.yml`. Do not create an unrelated workflow that publishes images without waiting for the tests.

1. **Keep the existing `tests` job** and its four test groups:
   - `tests/unit/`
   - `tests/integration/`
   - `tests/api/`
   - `tests/e2e/`
2. Keep the `pull_request` event for PRs targeting `main`.
3. **Also add `push` for `main`**, so merging a PR starts a run on the updated main branch.
4. Add a separate job named **`publish`** on an Ubuntu runner.
5. Make `publish` depend on the `tests` job. It must run **only** for a successful `push` to `main`, never on a pull request.
6. In `publish`, check out the repository, prepare Docker Buildx, log in to Docker Hub using the GitHub secrets, build the repository's Dockerfile, and push an image tagged:

   ```
   YOUR_DOCKER_ID/cars-crud:latest
   ```

7. Keep the workflow's GitHub token permissions minimal (read access to repository contents is sufficient for this example). A failed test must prevent publishing.

6. Hints (not a full solution)

Hint A: Configure the two events

This replaces the PR-only trigger from the previous exercise:

```
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
```

For `pull_request`, `main` is the **target branch**. For `push`, `main` is the branch receiving commits.

Hint B: Add a dependent job

Add `publish` **under `jobs:` at the same indentation as `tests:`**. Start from this unfinished outline:

```
  publish:
    needs: tests
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      # TODO: check out the code
      # TODO: set up Docker Buildx
      # TODO: authenticate to Docker Hub
      # TODO: build and push the image
```

`needs: tests` means this job waits for a successful `tests` job. The `if` condition ensures the job does not use publishing credentials on pull requests. **Do not remove the tests from the existing job.**

Hint C: Choose the actions

The relevant reusable actions are:

Action	Purpose
`actions/checkout@v7`	Make this repository available in the job.
`docker/setup-buildx-action@v4`	Prepare the Docker image builder.
`docker/login-action@v4`	Log in to Docker Hub using stored credentials.
`docker/build-push-action@v7`	Build a Dockerfile image and push its tag to a registry.

For `docker/login-action`, investigate its `with: username:` and `with: password:` options. GitHub secrets are read using this expression format:

```
${{ secrets.DOCKERHUB_TOKEN }}
```

For `docker/build-push-action`, investigate these `with:` options:

```
context: .
push: true
tags: YOUR_DOCKER_ID/cars-crud:latest
```

Replace the fixed `YOUR_DOCKER_ID` with the configured username (a GitHub secret expression is acceptable). **Do not** use `--build-arg` or `ENV` in the Dockerfile to pass registry passwords or tokens. You do not need to install Python or Playwright again in the `publish` job; the `tests` job already ran them.

Optional extra: In addition to `latest`, publish a tag containing the Git commit SHA, so you can identify exactly which commit produced the image.

7. Open a PR and verify both events

Save your changes, then commit and push your branch:

```
git add Dockerfile .dockerignore .github/workflows/tests.yml
git commit -m "Publish tested Cars CRUD image to Docker Hub"
git push -u origin docker-image-workflow
```

If there are no changes to `Dockerfile` or `.dockerignore`, Git will simply ignore them when staging.

1. On GitHub, open a pull request with **base** `main` and **compare** `docker-image-workflow`.
2. Open the **Checks** tab or **Actions** tab. Verify that unit, integration, API, and E2E tests pass.
3. Verify that `publish` **is skipped** for the PR, rather than logging in to Docker Hub.
4. Merge the PR into `main`.
5. Open the new workflow run triggered by the **push to** `main`. Confirm the test job finishes first, and then the publishing job succeeds.
6. Open your Docker Hub `cars-crud` repository and confirm the `latest` image tag was uploaded.

If branch protection is enabled, the successful test check can be required before the merge.

8. Prove the published image works

From a local terminal with Docker installed (replace `YOUR_DOCKER_ID`):

```
docker pull YOUR_DOCKER_ID/cars-crud:latest
docker run -d --name cars-from-hub -p 127.0.0.1:8000:8000 --mount
type=volume,source=cars-from-hub-data,target=/data YOUR_DOCKER_ID/cars-
crud:latest
```

Open http://localhost:8000/ and create a car. If port 8000 is already occupied by your earlier `cars-app` container, **stop that container first** (or use `127.0.0.1:8001:8000` for this run).

The application and HTML should behave just as they did when running the locally built image.

9. Completion checklist

- [ ] The Dockerfile builds the complete FastAPI + HTML application locally.
- [ ] The Docker Hub repository and GitHub secrets are configured.
- [ ] The workflow tests PRs targeting `main` without publishing.
- [ ] The workflow also runs tests on pushes to `main`.
- [ ] The publishing job depends on successful tests.
- [ ] Publishing runs only on `push` to `main`.
- [ ] Docker Hub contains `YOUR_DOCKER_ID/cars-crud:latest`.
- [ ] Pulling and running that image displays the Cars CRUD app.

10. Troubleshooting

Problem	What to check
Publishing job always skipped	Are you looking at the **push to main** run? Skipping on PRs is expected.
Publishing job runs after a failed test	Check the job-level `needs: tests` and remove any `always()` override.
`unauthorized` or `denied` during push	Verify repository name, Docker Hub token permission, and both GitHub secret names.
`Dockerfile not found`	Ensure `Dockerfile` is committed at the repository root and the build context is `.`.
No new run after merging	Check `on.push.branches`, YAML validity, and repository Actions settings.
`latest` is not updating	Check the tags passed to the build action and the Docker Hub repository page.
Browser cannot reach downloaded image	Check `docker ps`, `docker logs cars-from-hub`, and host port mapping.
Python base image pull is slow locally	Build with the Azure proxy using the `BASE_IMAGE` build argument in the Docker guide; the GitHub runner can use Docker Hub directly.
Azure proxy image cannot be found	Check the exact `/cache/library/python` path, tag, registry reachability, and access settings.

References

- Docker: GitHub Actions image build and push
- Docker login action
- Docker build and push action
- GitHub: Workflow syntax
- GitHub: Using secrets in Actions